2. Tokenization

CSC3043S Reasoning in AI: Large Language Models

Dr Jan Buys
University of Cape Town

Slides based on Jurafsky and Martin Chapter 2 and accompanying slides, available at https://web.stanford.edu/~jurafsky/slp3/



UNIVERSITY OF CAPE TOWN
IYUNIVESITHI YASEKAPA • UNIVERSITEIT VAN KAAPSTAD

Tokenization

- A language model processes a sequence of tokens $w_1, w_2, \ldots, w_n$
- Each token come from the vocabulary of the model $w_i \in V$ than can be represented as in index into the vocabulary list
- Tokenization is the task of converting a string of text into a sequence of token indices
- A good tokenizer should convert strings into a relatively compact sequence of tokens, but without making the vocabulary too large
  - Both of these influence efficiency and model expressiveness

Tokenization

Possible tokenizers:

- Character tokenizer
- Byte tokenizer
- Word tokenizer
- Subword tokenizer

Character tokenizer

- A raw unicode string is a sequence of unicode characters
- Each character can be converted into a code point integer

```
assert ord("a") == 97
assert ord("🌍") == 127757
```

- But there are about 150K unicode characters, and many are very rare
- So we get a low compression ratio (number of bytes per token), as well as an inefficient use of the vocabulary

Slide: Stanford CS336

Byte tokenizer

- Unicode strings can be represented as a sequence of bytes, which can be represented by integers between 0 and 255
- The most common Unicode encoding is UTF-8
- Some Unicode characters are represented by one byte, others take multiple bytes

```
assert bytes("a", encoding="utf-8") == b"a"

assert bytes("🌍", encoding="utf-8") == b"\xf0\x9f\x8c\x8d"
```

- Vocabulary size is small (256), but compression ratio is 1, so sequences will be very long

Slide: Stanford CS336

Word tokenizer

- Traditional NLP split strings into words
- Words are separated by spaces, but also need to handle punctuation and special characters
- In general, separate punctuation from words, but there are exceptions where punctuation is part of a word, e.g.
  - abbreviations (km/h, Ph.D., AT&T), prices (R45,55), dates (05-04-2021),
  - clitics (we're), multi-word expressions ($37-a-share)
- The simplest approach is to keep alphanumeric characters together and separate everything else
- Traditional tokenizers use more complex regular expressions

Word tokenizer

Advantages:

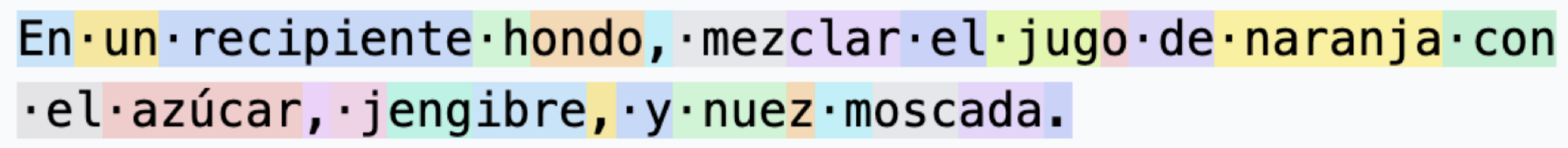
- Each token is meaningful
- Compression ratio is good

Disadvantages:

- The vocabulary size can be huge
- No matter how big the tokenizer's (fixed) vocabulary, there is always the possibility of processing a word not seen during training
  - The traditional approach was to represent such words with a special UNK token, but this removes information and limits the model
- The complexity of words vary between languages (e.g. isiZulu vs Sesotho)

Subword tokenizer

- Instead of having fixed rules of what a token is, train a tokenizer on raw text to construct a vocabulary tailored to the data
- The vocabulary size (which determines the compression ratio) can be set as a hyperparameter
- Intuition: represent common sequences of bytes (common words) by a single token; rare sequences are represented by many tokens



Subword tokenization

- Represent text as a sequence of *subword* tokens
- Subword tokens are the sub-parts/pieces of whole words.
- Choose the subword vocabulary **V** such that any string can be split into subwords in **V**

E.g.

```
This is a newly spoken sentence.
```

->

```
Th is is a new ly spok en se ntence .
```

Subword tokenization

Why?

- Handle unknown words as a sequence of known subwords.
  - e.g. newwebsite.com -> new website .com instead of [UNK]

- Compose the meaning of words from subwords (morphemes)
  - e.g. If “dog” has only been seen in singular form in training, but “-s” has been seen with other plural words, “dogs” can be composed as “dog” + “s”.

- Handles morphologically complexity - subwords may correspond to morphemes, improving modelling for morphologically rich languages

Byte-pair encoding (BPE)

- Use the data to tell use us how to tokenize
  - Ideally adapt to the properties of the language automatically
- Different subword tokenization algorithms have been proposed:
  - **Byte-Pair Encoding (BPE)** (Sennrich et al., 2016)
  - **Unigram language modeling tokenization** (Kudo, 2018)
  - **WordPiece** (Schuster and Nakajima, 2012)
- BPE is the most widely used, although Unigram tokenization can lead to better performance in some cases
- BPE was first proposed for data compression (Gage 1994), later adapted for neural machine translation, now widely used in LLMs

Byte-pair encoding

- BPE and related algorithms have two parts:
  - A **type learner** that takes a raw training corpus and induces a vocabulary (a set of types)
  - A **token segmenter** that takes a raw test sentence and tokenizes it according to that vocabulary

Byte-pair encoding

Type learner:

- Start with a vocabulary consisting of all individual characters and represent the corpus as sequences of items from this vocabulary

  = {A, B, C, D,…, a, b, c, d….}

- Repeat until *k* merges have been done:
  - Choose the two symbols that are most frequently adjacent in the training corpus (say 'A', 'B')
  - Add a new merged symbol 'AB' to the vocabulary
  - Replace every adjacent 'A' 'B' in the corpus with 'AB'

Byte-pair encoding

function BYTE-PAIR ENCODING(strings C, number of merges k) **returns** vocab V

$V \leftarrow$ all unique characters in C # initial set of tokens is characters
for $i = 1$ **to** k **do** # merge tokens k times
  $t_L, t_R \leftarrow$ Most frequent pair of adjacent tokens in C
  $t_{NEW} \leftarrow t_L + t_R$ # make new token by concatenating
  $V \leftarrow V + t_{NEW}$ # update the vocabulary
  Replace each occurrence of t_L, t_R in C with t_{NEW} # and update the corpus
return V

Byte-pair encoding

- The first step in applying a tokenizer is pre-tokenization
  - Typically split the string by spaces, separate punctuation, possibly normalize the text encoding
  - This enables implementation using more effective data structures
- BPE is then applied to the initial tokens
- To let the algorithm represent boundaries between tokens, a special start-of-word symbol '_' is added after spaces in the training corpus (or alternatively to indicate the end of words)

Byte-pair encoding

This is a newly spoken sentence.

(add special tokens)

_This _is _a _newly _spoken _sentence .

(apply BPE)

_Th is _is _a _new ly _spok en _se ntence .

Byte-pair encoding

- BPE example on a toy corpus

corpus	
5	l o w _
2	l o w e s t _
6	n e w e r _
3	w i d e r _
2	n e w _

Byte-pair encoding

- BPE example on a toy corpus
- The initial vocabulary is the set of characters

corpus	vocabulary
5 l o w _	_, d, e, i, l, n, o, r, s, t, w
2 l o w e s t _	
6 n e w e r _	
3 w i d e r _	
2 n e w _	

Byte-pair encoding

- Example: The most frequent pair is e r, so merge to er

corpus	vocabulary
5 l o w _	_, d, e, i, l, n, o, r, s, t, w, er
2 l o w e s t _	
6 n e w er _	
3 w i d er _	
2 n e w _	

Byte-pair encoding

- Next merge er _ to er_

corpus		vocabulary
5	l o w _	_, d, e, i, l, n, o, r, s, t, w, er, er_
2	l o w e s t _	
6	n e w er_	
3	w i d er_	
2	n e w _	

Byte-pair encoding

- Next merge n e to ne

corpus	
5	l o w _
2	l o w e s t _
6	ne w er_
3	w i d er_
2	ne w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w, er, er_, ne

Byte-pair encoding

- Next merges:

Merge	Current Vocabulary
(ne, w)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new
(l, o)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo
(lo, w)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low
(new, er_)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low, newer_
(low, _)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low, newer_, low_

Byte-pair encoding

- **Segmenter** algorithm (apply on dataset other than training corpus): Run each merge learned from the training data **greedily, in the order** they were learned (test frequencies don't play a role)

- So merge every e r to er, then merge er _ to er_, etc.
- Result:
  - Test set word "n e w e r _" would be tokenized as a full word
  - Test set word "l o w e r _" would be two tokens: "low er_"

Byte-pair encoding

- In practice, the most frequent words are included in the vocabulary, so not split into subwords, while rare words are split up
- Subwords may or may not correspond to morphemes
- We don't have to deal with the out-of-vocabulary word problem anymore, as all words can be tokenized into subwords, as long as their characters/bytes are included.

```
_jskbnckajs
(apply BPE)
_j sk b n ck a j s
```

Best practices in tokenization

- Perform as little lossy normalizations as possible
- For most tasks, do not lowercase or remove punctuation
- To support a wide range of characters, perform unicode normalization or use byte-level encoding
- Subword tokenization is standard: BPE is most commonly used, but Unigram has been shown to perform better in some cases
- A widely used library is Hugging Face's Tokenizers and OpenAI's tiktoken
- For large-scale tokenization use a library with an efficient C++ or Rust implementation